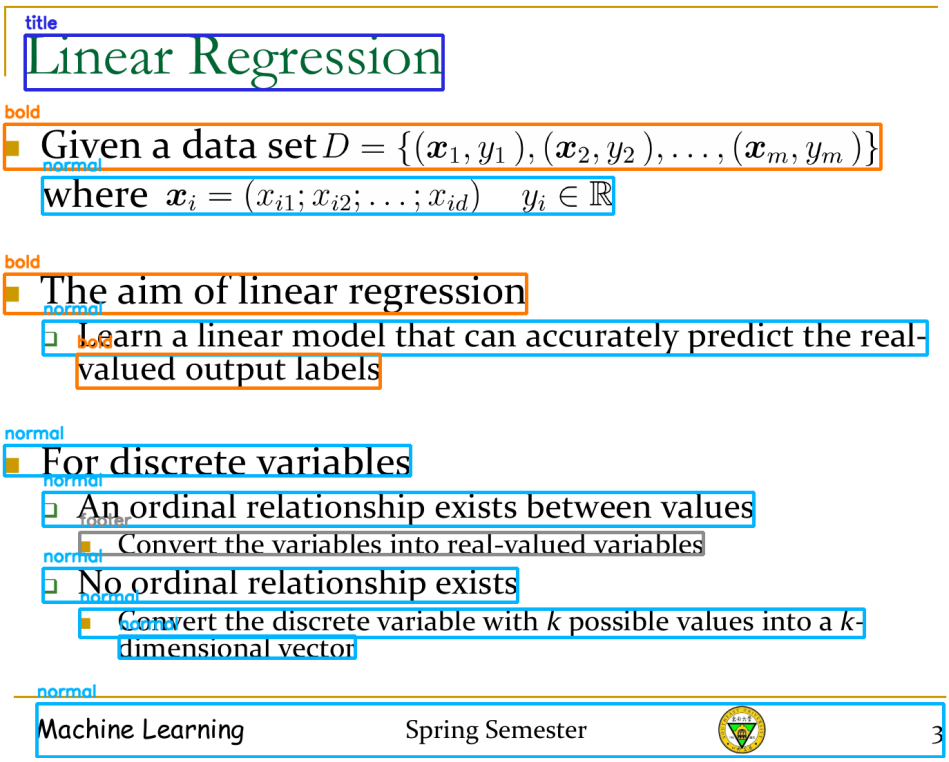
深度研究报告

Lecture2Notes Slide Structure Analysis 深度研究

文档、代码、论文与实跑 case 的联合拆解，以及它对现有 video-note pipeline 的可迁移价值

Codex

2026-03-26



核心问题：Lecture2Notes 的 Slide Structure Analysis 到底如何工作？它在真实实跑中的优势和断点是什么？其中哪些思想值得迁移到我们当前的视频笔记可视化链路里？
研究范围：聚焦 SSA 本身及其上下游接口：官方文档、仓库实现、论文描述、与本地实跑 case。默认以英文 slide-heavy 场景评估该模块，再讨论其对中文视频笔记 pipeline 的迁移边界。
目标读者：正在维护或设计视频讲义 / PDF 生成 pipeline 的工程人员，尤其是关心 slide-heavy 模式优化的人
证据窗口：外部资料访问与本地实跑均截至 2026-03-26

目录

1 执行摘要 **3**

2 研究范围与方法 **3**
2.1 本报告回答什么问题 . . . 3
2.2 证据来源 . . . 4
2.3 为什么 case 选英文 slide，而不是我们的中文视频 . . . 4

3 Lecture2Notes 全链路里，SSA 在哪里 **4**
3.1 SSA 不是前置定位器，而是后置结构化器 . . . 4
3.2 这件事对我们为什么重要 . . . 5

4 SSA 的工作原理 **5**
4.1 输入、输出与产物 . . . 5
4.2 OCR 层：词级别 Tesseract，保留位置与层级元数据 . . . 5
4.3 stroke width：它不是 OCR 置信度，而是字体粗细的近似代理 . . . 6
4.4 从词到行：通过全局 line 编号恢复可分类单元 . . . 6
4.5 行分类：一个非常硬的阈值逻辑 . . . 6
4.6 title detection：完全依赖“第一页第一段”的几何假设 . . . 7
4.7 spell check：论文说得很强，代码落点却很弱 . . . 7

5 源码层面的关键观察 **8**
5.1 它依赖的前提，比文档看起来更强 . . . 8
5.2 它对现代环境并不稳，需要兼容修正 . . . 8
5.3 原项目环境相当旧 . . . 8

6 实跑 case **9**
6.1 Case 选择 . . . 9
6.2 运行设置 . . . 9
6.3 Case 总体结果 . . . 10
6.4 三个最关键的页面观察 . . . 10
6.4.1 封面页：边框被 OCR 成伪文本，导致标题识别失败 . . . 10
6.4.2 普通 outline 页：大纲项被很好区分成强调与正文，但标题没有被识别为 title . 11
6.4.3 公式页：标题终于识别成功，但正文中很多数学行被抬升成 bold . . . 11
6.5 结构化输出的 downstream 影响 . . . 12
6.6 spell-check 的真实影响 . . . 12

7 Lecture2Notes SSA 值得借什么，不值得借什么 **12**
7.1 最值得借的，不是 Tesseract，而是“结构化 slide transcript” . . . 12
7.2 最不该借的，是旧 OCR 栈与硬编码阈值 . . . 13
7.3 对我们 pipeline 的可迁移价值 . . . 13
7.3.1 概念层：值得迁移 . . . 13
7.3.2 实现层：需要现代化重写 . . . 13
7.4 它具体能优化我们哪个部分 . . . 14
7.5 它明确帮不了我们的部分 . . . 14

8 建议的迁移方案 **14**
8.1 P0：先做轻量版结构化 slide transcript 14
8.2 P1：接入更强 layout 14
8.3 P2：让 agent / VLM 做纠偏，而不是做主干 15

9 结论与后续问题 **15**
9.1 已经有充分证据支持的结论 15
9.2 大概率正确但仍值得继续验证的推断 15
9.3 下一步最值得做的两件事 15

A 来源清单 **15**

B 本次为了跑通做的最小兼容修正 **17**

1 执行摘要

一句话判断

Lecture2Notes 的 SSA 不是一个“理解整段视频”的模块，而是一个建立在**已提纯唯一 slide 图像集合**之上的 **OCR 后处理与轻量版式分类器**。它最有价值的思想不是 Tesseract 本身，而是把 slide 转成带有 `title` / `bold` / `normal` / `footer` 语义的结构化中间层；它最明显的局限，则是对 OCR 噪声、英文词典、硬编码阈值和页面模板非常敏感。

- 在 Lecture2Notes 的完整链路里，SSA 排在 slide classifier、black border removal / perspective crop、clustering 去重之后。也就是说，它默认输入已经是“最佳 slide 样本”，而不是任意视频帧。
- SSA 的核心算法非常简单：用 Tesseract 提取词级 OCR 与 bounding box，计算每个词的 stroke width，再按行聚合，用相对均值阈值把每行分类成 `bold`、`normal`、`footer`，并用另一套规则识别第一页第一段是否是 `title`。
- 论文和文档都说“Tesseract 输出会被 SymSpell spell check”，但代码实际只对 `slide-ocr.txt` 这种原始文本做了 spell check；用于下游结构化摘要的 `slide-ssa.json` 仍保留原始 OCR 错误。
- 我在当前机器上实际跑了它的 SSA 核心，对一个公开英文 slide deck 的前 8 页做了测试。结果显示：8 页中只有 1 页成功识别出 `title`；封面页因为上方装饰线被 OCR 成“re“，导致标题识别失效；普通 outline 页里的 footer 没被稳定过滤；技术词和数学记号在 SymSpell 后反而被破坏。
- 因此，SSA **不适合作为我们 pipeline 的通用主干**，但其“把 slide OCR 提炼为结构化中间表示”的思路非常值得借。对于我们的系统，更合理的路线是：只在 `slide-heavy` / `static-outline` / `screen-demo` 模式下启用一种**现代化 SSA**，换用更强的 OCR / layout 能力，并把产物写成 `structured_slide.json` 或 `layout_transcript.json`。

最重要的迁移结论

如果直接把 Lecture2Notes 的 Tesseract + stroke-width heuristics 原样搬进来，在中文、多公式、UI、白板、评论视频里会迅速失效。应该迁移的是中间层思想，而不是旧 OCR 栈和硬编码阈值。

2 研究范围与方法

2.1 本报告回答什么问题

本报告聚焦四个问题：

1. Lecture2Notes 的 SSA 在整个系统中扮演什么角色？
2. 它的算法、代码路径、输入输出结构到底是什么？
3. 在当前机器和一个真实 case 上跑起来之后，它表现如何，暴露出哪些断点？
4. 它能否优化我们当前的 video-note pipeline，如果能，应该借什么、不借什么？

2.2 证据来源

证据组成

- 官方资料：Lecture2Notes 文档、README、论文 PDF。
- 源码：本地 clone 的 `slide_structure_analysis.py`、`spell_check.py`、`summarizer_class.py`、`summarization_approaches.py`。
- 实跑：选取一个公开英文 slide deck，将前 8 页导成 PNG，直接调用 SSA 核心进行分析。
- 环境审查：检查原项目 `environment.yml` 与当前机器的二进制 / Python 包差异，定位兼容性断点。

2.3 为什么 case 选英文 slide，而不是我们的中文视频

这是一个刻意选择。原因有三点：

- Lecture2Notes 的 SSA 文档、SymSpell 词典和论文实验默认都是英文 slide-heavy 语境；
- 当前机器默认没有英文 `eng` Tesseract 语言包，我是手动补的；如果再直接测中文，结果会被语言包问题污染；
- 我们此处要回答的是“模块原理与迁移价值”，所以应先在它最擅长的语境里评估，再讨论迁移边界。

3 Lecture2Notes 全链路里，SSA 在哪里

3.1 SSA 不是前置定位器，而是后置结构化器

官方文档和 README 都把端到端流程描述为：

1. 抽帧
2. slide classifier 区分 `slide` 与 `presenter_slide`
3. black border removal / perspective crop
4. clustering 去重并挑出最佳 slide 样本
5. 对唯一 slide 样本运行 SSA
6. figure detection
7. ASR / transcript
8. 组合 slide transcript 与音频 transcript，再做 summarization

这意味着 SSA 的前提非常强：它不是拿来解决“视频里哪里有关键图”的，而是拿来解决“我已经有一组不错的 slide 图片了，现在如何把它们转成结构化文本”。

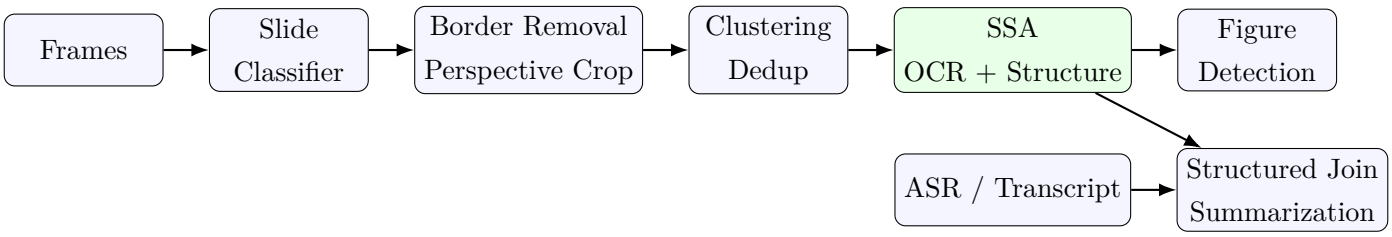


图 1: Lecture2Notes 中 SSA 的位置：它服务于“已去重 slide 集合”的结构化，而不是服务于视频关键帧搜索

3.2 这件事对我们为什么重要

它直接决定了是否值得迁移。我们当前 pipeline 最大的痛点之一，是还没稳定产出“最终可解释的 figure plan 与 frame manifest”。Lecture2Notes 的 SSA 并不能帮我们解决“如何从视频定位图”，但它能启发我们在 slide-heavy 模式下增加一层：

$$\text{候选 slide 图像} \rightarrow \text{结构化 slide transcript}$$

这对章节标题、强调行、正文结构、footer 过滤都很有帮助。

4 SSA 的工作原理

4.1 输入、输出与产物

代码层面，SSA 的主入口是：

- `analyze_structure(image, ...)`
- `all_in_folder(path, ...)`
- `write_to_file(raw_texts, json_texts, ...)`

其中：

- 输入是一张 slide 图像，或一个装有 slide 图像的文件夹；
- 输出有两类：一类是原始 OCR 合并文本；一类是结构化 JSON，其中按行保存文本及其类别；
- 如果在完整 Summarizer 流程里运行，还会额外写出：`slide-ocr.txt` 和 `slide-ssa.json`

4.2 OCR 层：词级别 Tesseract，保留位置与层级元数据

SSA 首先调用 Tesseract 的 `image_to_data`，拿到词级别输出。与纯 `image_to_string` 不同，这里保留了：

- `block_num`
- `par_num`
- `line_num`
- `word_num`

- 每个词的 `left, top, width, height`

这个设计很关键。因为 SSA 后面并不是直接按词输出，而是要恢复“哪一堆词属于同一行、同一段”，再对行级对象做分类。

4.3 stroke width：它不是 OCR 置信度，而是字体粗细的近似代理

SSA 最独特的地方，是它没有只依赖 bounding box 高度，而是额外计算了每个词图像块的 stroke width。其流程可写成：

$$\mathrm{SW}(w) = \mathrm{mean}\left(\mathrm{PeakLocalMax}\left(\mathrm{DT}\left(\mathrm{OtsuBin}(w)\right)\right)\right)$$

- w：某个词的图像块
- OtsuBin：Otsu 阈值分割，并在代码中做反色二值化
- DT：距离变换
- PeakLocalMax：对距离图取局部峰值
- $\mathrm{SW}(w)$：这些峰值的平均值，作为词的笔画粗细近似

直觉上，标题和粗体文本通常拥有更大的 stroke width，而页脚和小字则更小。

4.4 从词到行：通过全局 line 编号恢复可分类单元

SSA 会扫描 OCR DataFrame，并用一个简单规则重建 `global_line_num`：

- 每当 `word_num == 1`，就认为开启了一条新行；
- 然后对同一 `global_line_num` 的词做聚合；
- 文本列做字符串拼接，其余数值列求均值。

因此，真正被分类的对象不是词，也不是段落，而是**行**。

4.5 行分类：一个非常硬的阈值逻辑

在代码里，行分类分为四类：

- `2`: title
- `1`: subtitle / bold
- `0`: normal
- `-1`: small / footer

对非 title 行，核心分类规则可以写成：

$$\mathrm{cat}(l) = \begin{cases} 1, & s_l > \bar{s}(1+\gamma) \ \vee \ h_l > \bar{h}(1+\beta) \\ -1, & s_l < \bar{s}(1-\gamma) \ \wedge \ h_l < \bar{h}(1-\beta) \\ 0, & \text{otherwise} \end{cases}$$

- l：某一文本行
- s_l：该行平均 stroke width
- h_l：该行平均 bounding box height
- $\bar{s}$：整页词的平均 stroke width
- $\bar{h}$：整页词的平均高度
- γ：stroke width 阈值，默认 0.1
- β：height 阈值，默认 0.2

这个规则有两个鲜明特征：

- 它对 `bold` 很宽松，只要“stroke 宽”或“字高高”任一成立即可；
- 它对 `footer` 很严格，必须两个条件同时偏小。

这解释了为什么它宁愿把一些正文误判成 bold，也不愿轻易把正文误删成 footer。

4.6 title detection：完全依赖“第一页第一段”的几何假设

title detection 不是机器学习，而是另一套规则：

1. 必须是第一 block 的第一 paragraph；
2. 其平均 `top` 在页面上三分之一；
3. 其平均 `left` 小于页面宽度的 77%；
4. 字符数超过 3；
5. 平均 stroke width 高于全页均值加一个标准差；
6. 平均字高高于全页平均字高。

如果整页只有一个 block / paragraph，则第 5 和第 6 条被关闭，以免“整页平均”退化成标题本身。

SSA 的本质

SSA 不是一个学习型版面分析模型，而是一套“词级 OCR + 词图像 stroke width + 行级阈值分类 + 第一段标题规则”的启发式系统。

4.7 spell check：论文说得很强，代码落点却很弱

官方文档和论文都写了“All Tesseract outputs are spell checked with SymSpell”。但实际代码路径是：

- `slide_structure_analysis.all_in_folder()` 产出 `raw_texts` 和 `json_texts`
- `summarizer_class.step_slide_structure_analysis()` 只对 `raw_texts` 调用 `SpellChecker.check_all()`

- 随后把**已 spell-check 的 raw text** 与**未 spell-check 的 json_texts** 一起写盘

这意味着：

- `slide-ocr.txt` 可能是拼写纠正后的；
- 但 `slide-ssa.json` 里保存的结构化文本，仍然保留原始 OCR 错误；
- 下游 `structured_joined_sum()` 读取的是 SSA JSON，所以它实际上消费的是**未纠正 OCR**。

5 源码层面的关键观察

5.1 它依赖的前提，比文档看起来更强

从 `end_to_end/general.rst` 和 `summarizer_class.py` 可以看到，SSA 默认处理的是聚类后最佳 slide 样本目录 `best_samples_dir`。因此它对输入假设非常强：

- 图像已被裁成只含 slide 主体；
- 重复页基本去掉；
- 排序和 frame number 语义稳定；
- 语言基本是英文；
- 模板接近传统课堂 slides。

5.2 它对现代环境并不稳，需要兼容修正

在当前机器上，我为了跑通 SSA 做了两个极小兼容补丁：

1. `slide_structure_analysis.py`：为空的 stroke width 峰值返回 0，并把 `grouped.mean()` 改成 `grouped.mean(numeric_only=True)`，否则新版 pandas 会报 `dtype 'str' does not support operation 'mean'`。
2. `spell_check.py`：不再使用老式的 `pkg_resources.resource_filename`，而是直接从 `symspellpy` 包目录定位资源文件。

这两个问题说明：**SSA 的思想仍可用，但原仓库并不是“拿来即跑”的现代可维护组件**。

5.3 原项目环境相当旧

原始 `environment.yml` 还带着：

- 旧版 PyTorch / TensorFlow
- conda 环境假设
- Vosk / DeepSpeech / youtube_dl / pytorch_lightning 老组合
- 默认强依赖英语 NLP 生态

这对研究复现没问题，但对直接抽取 SSA 作为一个现代微模块不友好。

6 实跑 case

6.1 Case 选择

本次 case 选择公开 slide deck：

- 来源：https://seuml.github.io/ml-spring-2024/assets/slides/lecture2.pdf
- 标题：`Lecture 2: Linear Regression`
- 页数：70 页
- 本次只取前 8 页，模拟“已去重后的最佳 slide 样本”

6.2 运行设置

- 本地 clone commit：`10210930ac4c3ffe3de7de0a36b74514122ff445`
- Python：系统 `Python 3.14.3`，新建最小 venv
- OCR：系统 `tesseract`，额外下载 `eng.traineddata`
- 只安装 SSA 所需最小依赖：`numpy`，`pandas`，`pytesseract`，`scikit-image`，`opencv-python-headless`，`tqdm`，`pillow`，`symspellpy`

```
python3 -m venv artifacts/venv
source artifacts/venv/bin/activate
pip install numpy pandas pytesseract scikit-image \
  opencv-python-headless tqdm pillow symspellpy

curl -L https://raw.githubusercontent.com/tesseract-ocr/tessdata_fast/main/eng.traineddata \
  -o artifacts/tessdata/eng.traineddata

pdftoppm -f 1 -l 8 -png case/input_pdf/lecture2.pdf case/slides/tmp
# 重命名为 slide_0001.png ... slide_0008.png

TESSDATA_PREFIX=artifacts/tessdata \
PYTHONPATH=artifacts/lecture2notes-repo \
python -c 'from lecture2notes.end_to_end import slide_structure_analysis; ...'
```

Listing 1: 本次 case 的最小运行步骤

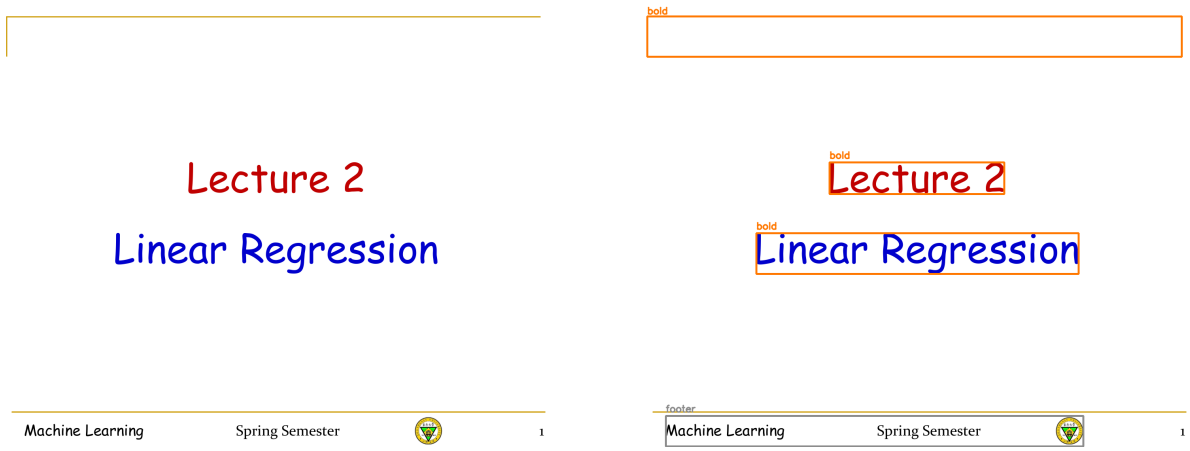
6.3 Case 总体结果

表 1: 8 页 case 的总体统计

指标	含义	结果
输入页数	抽取并送入 SSA 的 slide 数	8
平均每页行数	`slide-ssa.json` 中每页保留下来的行数	8.62
category=2	title 行数量	1
category=1	bold / subtitle 行数量	32
category=0	normal 行数量	28
category=-1	footer / small 行数量	8
spell-check 影响	对 `slide-ocr.txt` 进行 SymSpell 修正	有，但会损伤技术词

6.4 三个最关键的页面观察

6.4.1 封面页：边框被 OCR 成伪文本，导致标题识别失败

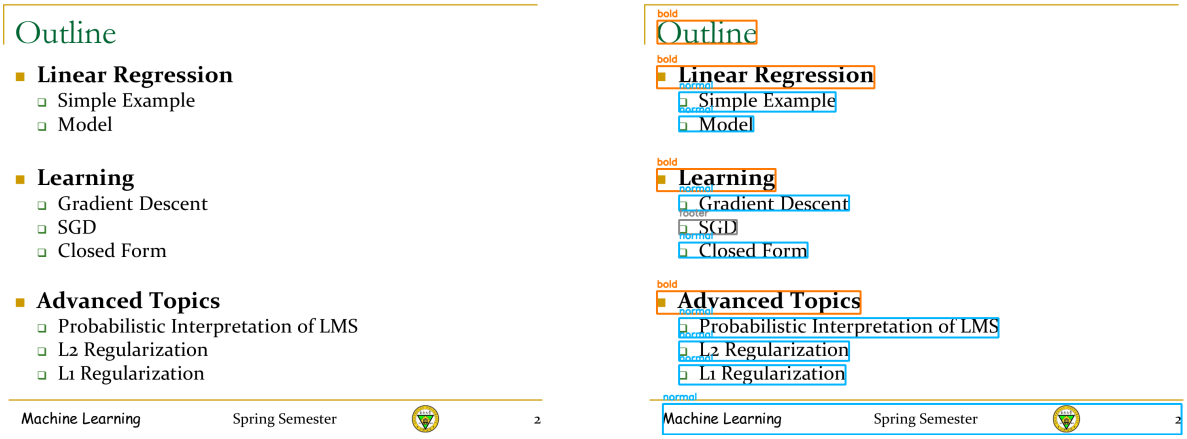


(a) 原始 slide　　(b) SSA 行分类结果

图 2: 封面页的顶部装饰线被 OCR 成一条伪文本行 “re“，因此第一 block 不再是实际标题，导致 title detection 失效

调试结果表明，Slide 1 的第一 block 文本是 “re“，字符数只有 2，不满足 title check 的字符门槛，因此真正的 “Lecture 2“ 和 “Linear Regression“ 都只被判成 `bold`，最终 structured output 的标题退化成默认值 `Slide 1`。

6.4.2 普通 outline 页：大纲项被很好区分成强调与正文，但标题没有被识别为 title



(a) 原始 outline 页　　(b) SSA 结果

图 3: Slide 2 说明了 SSA 的优点与缺点并存：大纲级别能大体分出来，但页标题 ‘Outline‘ 仍未被提升为 title，页脚也没有被完全过滤

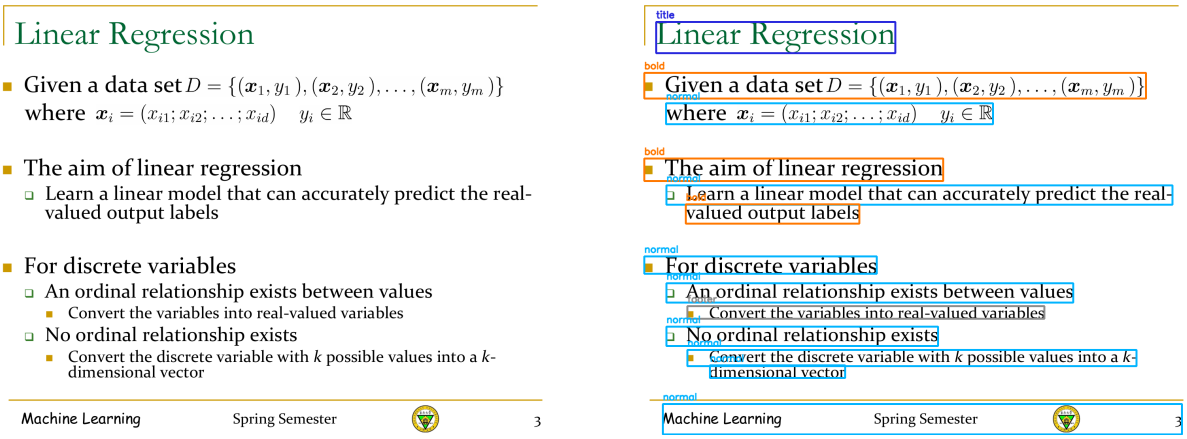
对 Slide 2，SSA 的优点是明显的：

- `Linear Regression`、`Learning`、`Advanced Topics` 这类大纲层级被判成 `bold`
- 子项大多被判成 `normal`

但它的缺点也同样明显：

- 顶部标题 `Outline` 因 stroke width 不够大，没有进入 `title`
- 页脚 `Machine Learning Spring Semester ...` 被保留成 `normal`
- `SGD` 被误判成 `footer`

6.4.3 公式页：标题终于识别成功，但正文中很多数学行被抬升成 bold



(a) 原始 slide　　(b) SSA 结果

图 4: Slide 3 是 8 页里唯一成功 title detection 的页面，但也显示了“高字高 / 高 stroke 容易把正文推成 bold”的副作用

这一页里，`Linear Regression` 正确被识别为 `title`。但正文中的公式和章节句首也经常因为行高更大而被判成 `bold`。这对“做 outline”未必是坏事，但对“忠实恢复原 slide 结构”就已经偏粗糙了。

6.5 结构化输出的 downstream 影响

把 `slide-ssa.json` 按 `structured_joined_sum()` 的逻辑消费后，前 8 页的 slide 标题大致会变成：

表 2: 前 8 页 slide 的标题产出结果

页码	实际视觉标题	下游生成标题
1	Lecture 2 / Linear Regression	Slide 1
2	Outline	Slide 2
3	Linear Regression	Linear Regression
4	Linear Regression	Slide 4
5	Linear Regression example	Slide 5
6	Linear Regression example	Slide 6
7	Linear Regression example	Slide 7
8	Linear Regression	Slide 8

也就是说，在本次 case 里，**8 页只有 1 页能够把视觉标题真正下沉为结构化标题键**。这对后续生成“按 slide 标题组织的结构化讲义”显然是不够稳的。

6.6 spell-check 的真实影响

一次很有代表性的 before / after

未 spell-check：

```
... Probabilistic Interpretation of LMS ... L2 Regularization ... Li Regularization ...
```

经 SymSpell 后：

```
... probabilistic interpretation of los ... la regularization ... i regularization ...
```

这说明 SymSpell 的确能修一些普通英语词，但对技术缩写、数学记号、层级标记、课程模板文本并不友好。对我们今天的视频笔记场景，尤其是中文、公式、代码、界面控件文本，这一步基本不能原样照搬。

7 Lecture2Notes SSA 值得借什么，不值得借什么

7.1 最值得借的，不是 Tesseract，而是“结构化 slide transcript”

Lecture2Notes 给我们的最大启发不是 OCR 引擎，而是 artifact 设计：

- 不要把 slide OCR 只当成一坨纯文本；
- 应该把它变成“按行组织的结构化表示”，并保留语义类别；
- 这样下游才能做：章节标题抽取、强调行提炼、footer 过滤、transcript 与 slide 的结构对齐。

这正对应我们当前 pipeline 的一个空白：字幕之后还缺一层能够把“图像里的文字结构”稳定描述出来的中间层。

7.2 最不该借的，是旧 OCR 栈与硬编码阈值

直接照搬 Lecture2Notes SSA 会遇到这些问题：

- 默认假设英语，且依赖英语词典 spell check；
- 依赖 Tesseract block / paragraph 的稳定性，而现代 slides 上 logo、装饰线、页码、导航条都可能打乱第一 block；
- 只用 stroke width 与字高做判断，无法处理 code block、公式、UI、表格、混合语言；
- footer 规则过死，必须两个尺度都偏小；
- 对 progressive reveal、screen-demo、白板视频几乎没有直接帮助。

7.3 对我们 pipeline 的可迁移价值

我建议把它拆成“概念层”和“实现层”来判断。

7.3.1 概念层：值得迁移

- 在 `slide-heavy / static-outline / screen-demo` 模式里，加入一层 `structured_slide.json`
- 该 JSON 至少记录：标题候选、强调行、正文行、footer / boilerplate、以及它们对应的图像区域
- 下游用它来：辅助章节命名、生成 callout、过滤 footer、对 PDF 正文做逻辑压缩

7.3.2 实现层：需要现代化重写

表 3: Lecture2Notes SSA 与我们更合理实现的对照

环节	Lecture2Notes 做法	更适合我们的做法
OCR	Tesseract 英文 OCR	PaddleOCR / docTR / 更强多语言 OCR
文本组织	词级 OCR 后按行聚合	保留行和区域，两层结构都留
类别判定	stroke width + height heuristics	几何特征 + OCR 置信度 + layout analysis + 可选 VLM 纠偏
title detection	第一 block / 第一 paragraph 规则	多候选标题区域 + 模式化校验，不只盯第一段
spell check	英文 SymSpell	按语言与模式选择，技术词不要默认纠正
适用模式	传统英文 slide-heavy lecture	只在 slide-heavy / static-outline / screen-demo 中按需启用

7.4 它具体能优化我们哪个部分

如果只谈 ROI，我认为它对我们现有系统最有帮助的是这三个点：

1. **章节结构提炼**

   对 slide-heavy 视频，slide 标题和大纲项常常比 transcript 更稳定。一个现代化 SSA 能帮助 outline 阶段更快得到章标题与小节骨架。

2. **图文规划**

   如果知道某页 slide 的“主标题、强调项、正文、页脚”分布，agent 在决定“放整页图还是只裁一块”时会更有依据。

3. **PDF 正文压缩**

   对 `static-outline` 和某些 screen-demo，结构化 slide transcript 能帮我们在 PDF 里更自然地用项目符号、强调块和表格重写，而不是依赖长段 OCR 原文。

7.5 它明确帮不了我们的部分

- 不能替代关键帧搜索：它不处理“从视频里哪里找图”
- 不能处理白板累积：它没有 progressive reveal 最终态搜索能力
- 不能处理 commentary：这类视频的高价值信息不在 slide 结构里
- 不能直接解决中文 / 公式 / UI 多样性：旧 OCR 与英文词典都不适配

对我们最实际的建议

不要引入原版 Lecture2Notes SSA；应该引入一层 **SSA-like artifact**。也就是在 slide-heavy 模式下，新增一个现代化的结构化版面转写模块，把候选 slide 图像变成可被大纲生成与 PDF 写作消费的中间表示。

8 建议的迁移方案

8.1 P0：先做轻量版结构化 slide transcript

- 只在 `slide-heavy` / `static-outline` 模式启用
- 对最终候选 slide 图运行 OCR
- 不做复杂分类，只先做三类：`title-candidate`、`body`、`footer-candidate`
- 产出 `structured_slide.json`

8.2 P1：接入更强 layout

- 用 PaddleOCR layout analysis 或类似模块代替手工几何阈值
- 保留文本块区域，辅助 crop 与 PDF 图注生成
- 在公式页、表格页、UI 页加专门 probe

8.3 P2：让 agent / VLM 做纠偏，而不是做主干

- 让确定性 OCR / layout 先产出候选结构
- 再让 VLM 判断哪个标题候选最像本页主题
- VLM 只做纠偏和排序，不直接取代底层 artifact

9 结论与后续问题

9.1 已经有充分证据支持的结论

- Lecture2Notes SSA 是一个后置结构化模块，不是视频定位模块。
- 它的核心是启发式：OCR + stroke width + height + 几何标题规则。
- 它最有价值的是“把 slide 文本结构化”，而不是具体的 OCR 栈。
- 在真实英文 case 上，它连 title detection 都不够稳；普通现代 slides 上，装饰线和页脚就足以破坏规则。
- 论文 / 文档与代码之间存在一处重要落差：spell-check 并没有真正进入 SSA JSON。

9.2 大概率正确但仍值得继续验证的推断

- 如果换成更强多语言 OCR + layout analysis，这种“SSA-like artifact”对我们的 slide-heavy 视频会非常有帮助。
- 对中文投资视频、读书视频、软件演示视频，结构化版面转写比纯 OCR 原文更可能提升 PDF 逻辑质量。
- 对 commentary 和 board-heavy 视频，这层的收益会明显下降，应通过 mode routing 控制。

9.3 下一步最值得做的两件事

1. 在我们现有 pipeline 里设计一个 `structured_slide.json`，先只面向 `slide-heavy / static-outline` 模式。
2. 用现代 OCR / layout 工具做一个小实验，对比“纯 OCR 原文”和“结构化 slide transcript”对 PDF 大纲质量、footer 过滤和标题抽取的改善幅度。

A 来源清单

Source ID	模态	标题 / 标签	链接或路径
W1	Web	Lecture2Notes SSA 文档	https://lecture2notes.readthedocs.io/en/latest/end-to-end/slide-structure-analysis.html
W2	Web	Lecture2Notes 文档总览	https://lecture2notes.readthedocs.io/en/latest/getting-started/about.html

Source ID	模态	标题 / 标签	链接或路径
W3	Web	Lecture2Notes 端到端说明	https://lecture2notes.readthedocs.io/en/latest/end-to-end/general.html
W4	Web	Lecture2Notes 论文 PDF	https://haydenhousen.com/media/lecture2notes-paper-v1.pdf
W5	Web	Yang et al. 2011 引用	http://ieeexplore.ieee.org/document/6120629/
W6	Web	实跑输入 PDF	https://seuml.github.io/ml-spring-2024/assets/slides/lecture2.pdf
W7	Web	Tesseract 英文语言包	https://raw.githubusercontent.com/tesseract-ocr/tessdata_fast/main/eng.traineddata
L1	Local	Lecture2Notes clone	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-lecture2notes-ssa-deepresearch/artifacts/lecture2notes-repo
L2	Local	clone commit	10210930ac4c3ffe3de7de0a36b74514122ff445
L3	Local	slide_structure_analysis.py	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-lecture2notes-ssa-deepresearch/artifacts/lecture2notes-repo/lecture2notes/end_to_end/slide_structure_analysis.py
L4	Local	spell_check.py	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-lecture2notes-ssa-deepresearch/artifacts/lecture2notes-repo/lecture2notes/end_to_end/spell_check.py
L5	Local	summarizer_class.py	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-lecture2notes-ssa-deepresearch/artifacts/lecture2notes-repo/lecture2notes/end_to_end/summarizer_class.py
L6	Local	summarization_approaches.py	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-lecture2notes-ssa-deepresearch/artifacts/lecture2notes-repo/lecture2notes/end_to_end/summarization_approaches.py
L7	Local	environment.yml	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-lecture2notes-ssa-deepresearch/artifacts/lecture2notes-repo/environment.yml
L8	Local	case slides	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-lecture2notes-ssa-deepresearch/case/slides
L9	Local	slide-ssa.json	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-lecture2notes-ssa-deepresearch/case/output/slide-ssa.json

Source ID	模态	标题 / 标签	链接或路径
L10	Local	slide-ocr-spellchecked.txt	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-lecture2notes-ssa-deepresearch/case/output/slide-ocr-spellchecked.txt
L11	Local	slide1-annotated.png	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-lecture2notes-ssa-deepresearch/figures/slide1-annotated.png
L12	Local	slide2-annotated.png	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-lecture2notes-ssa-deepresearch/figures/slide2-annotated.png
L13	Local	slide3-annotated.png	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-lecture2notes-ssa-deepresearch/figures/slide3-annotated.png

B 本次为了跑通做的最小兼容修正

- `slide_structure_analysis.py`
  - 空峰值列表时，`stroke_width` 直接返回 0.0
  - `grouped.mean()` 改为 `grouped.mean(numeric_only=True)`
- `spell_check.py`
  - 资源文件定位从 `pkg_resources` 改为直接从 `symspellpy` 包目录读取

这些修正只用于本地研究复现，不应被误解为 upstream 已经支持当前现代 Python / pandas 运行环境。